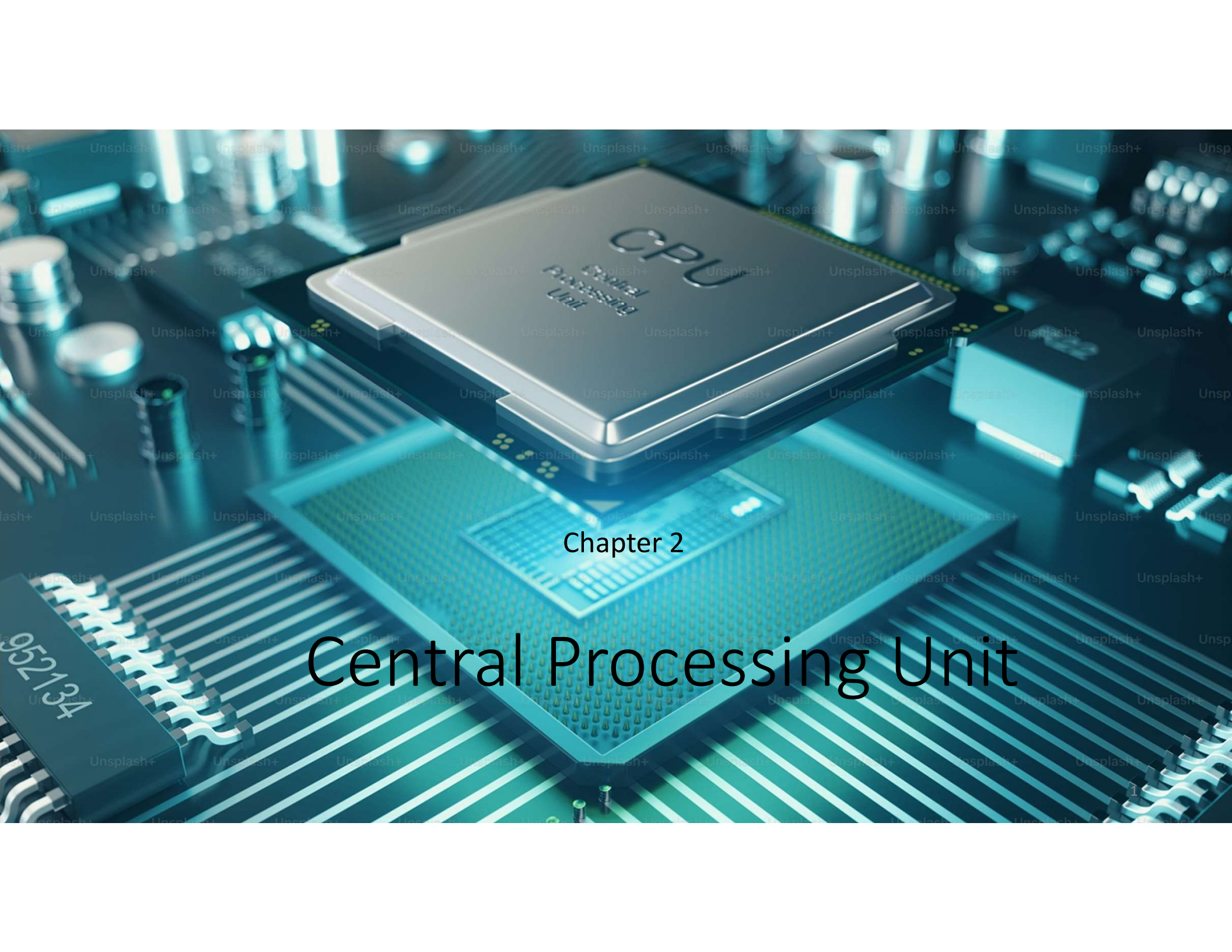




Chapter 2

Central Processing Unit

Contents

2 Central Processing Unit (CPU) (7 hours)

2.1 Processor bus organization

2.2 Processor register organization: Control word, examples of microoperations

2.3 Stack organization: Register stack, memory stack, reverse polish notation, evaluation of arithmetic expressions

2.4 Instruction formats: CPU organization, zero and more address instruction formats

2.5 Addressing modes: Types, examples, strengths and weaknesses

2.6 Instruction set

2.6.1 Data transfer instruction

2.6.2 Data manipulation instruction: Arithmetic, logical and shift operations

2.6.3 Program control instruction

2.7 Status bit conditions

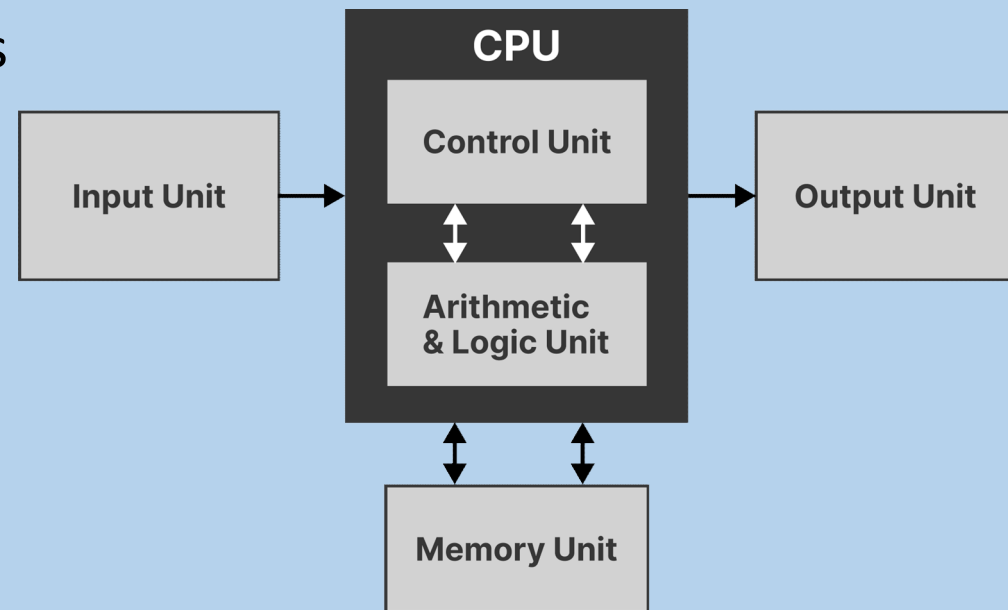
2.8 Interrupt: Definition, types, processing and ISR

Processor

- The **processor** or **Central Processing Unit (CPU)** is the brain of a computer.
- It executes instructions and controls the operations of other components.
- Processor organization refers to the **internal architecture** of the CPU, including its components and how they interact.
- A well-organized processor ensures **fast, efficient, and reliable** execution of tasks.

Component of Processor Organization

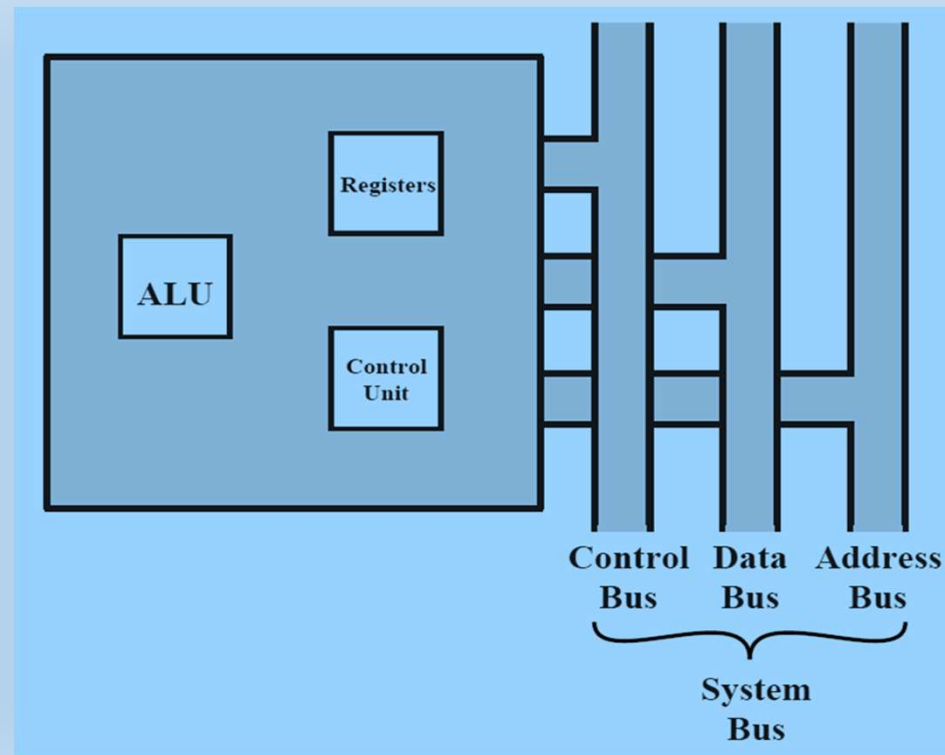
- **Control Unit:** It directs the operation of the CPU and manages the data.
- **Arithmetic Logic Unit (ALU):** It executes all arithmetic and logical operations.
- **Memory Unit (MU):** It stores and retrieves the execution data of the CPU.



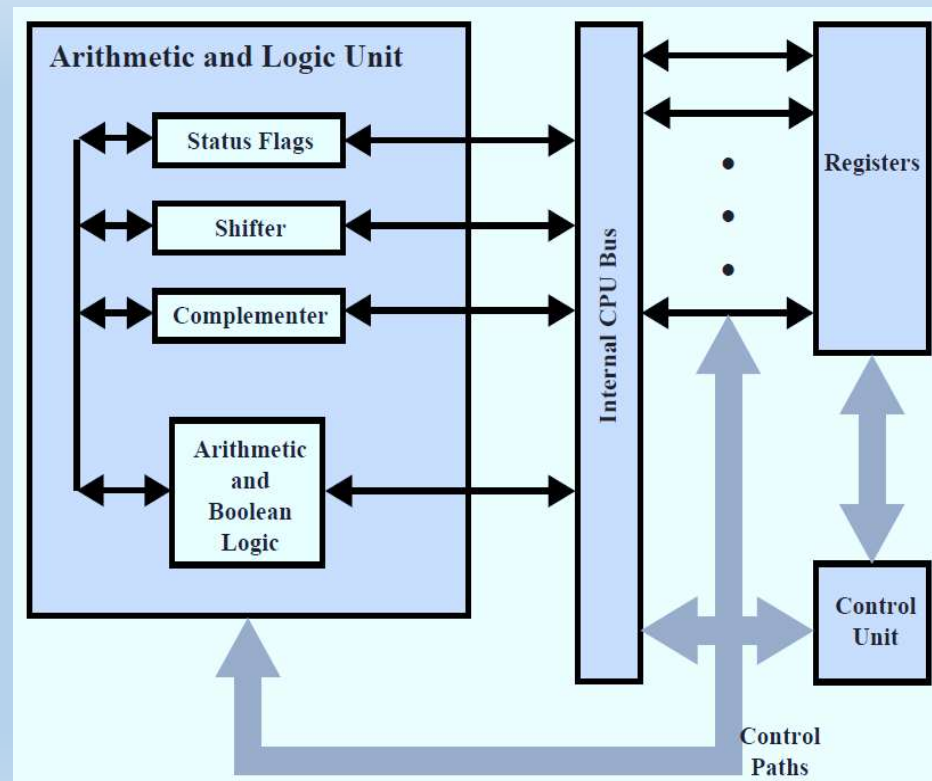
Processor Bus Organization

- Operations performed by processor in general:
 - Fetch instruction
 - Interpret instruction
 - Fetch Data
 - Process data
 - Write data
- Requires a way to store data temporarily – **Registers**
- Processing of data – **ALU**
- Control movement of data & its operations – **Control Unit**
- Transfer of data between registers and ALU – **Bus**

CPU with the System Bus



Internal Structure of the CPU

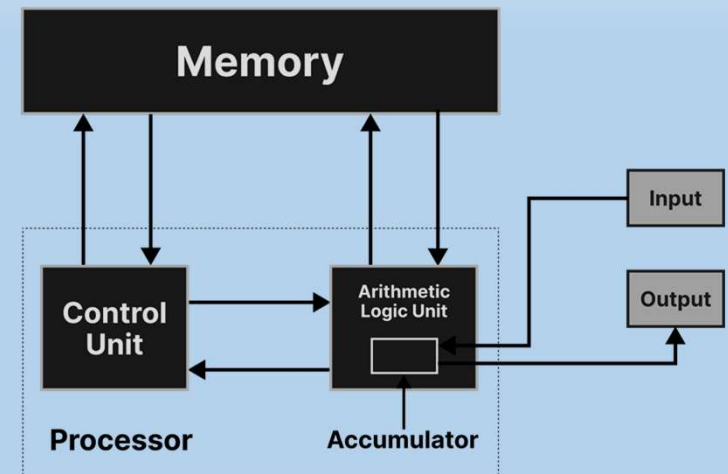


Types of CPU Organization in Computer Architecture

- There are 3 types of CPU Organization in Computer Architecture, they are:
 - Single Accumulator Based Organization
 - General Register Based Organization
 - Stack Organization

Single Accumulator Organization

- This is one of the simplest forms of CPU design. In this organization, there is only one accumulator register that holds intermediate data for arithmetic or logical operations.
- The CPU fetches data from memory into the accumulator, performs the operation, and stores the result back into the accumulator or memory.



Von Neumann Basic Structure

General Register Organization

- Register organization refers to how various registers are structured, categorized, and accessed in a CPU. Registers are fastest memory elements within the processor used to store and manipulate data during execution.
- Registers can be broadly classified into two categories:
 - User Visible register
 - Control and status register

User Visible register

- Registers that can be **directly accessed and used** by the programmer during program execution.
- The purpose of using it are:
 - To store operands and intermediate results.
 - Allow fast access to commonly used data.
- **Types:**
 - General-Purpose Registers (GPRs):Used for arithmetic, logic, and data manipulation. e.g., R0, R1, ..., R7 (in RISC); AX, BX in x86.
 - Data Registers: Hold numeric values or operands. Like A,B,C,D, H,L

Types (Contd...)

- **Address Registers:**
 - Store addresses used for memory access.
 - May include:
 - **Index registers** (for array addressing)
 - **Segment pointers**
 - **Stack pointer (SP)**
- **Condition Code/Flags Register** (sometimes user-visible):
 - Contains bits set by the CPU to reflect outcomes of operations:
 - Zero (Z), Carry (C), Negative (N), Overflow (V)

Control and Status Registers

- Registers used by the **Control Unit (CU)** and **Operating System** to monitor and control processor operations.
- Help manage and control **instruction execution, interrupts, and system-level events.**

Types(Control and status register)

Type of Register	Purpose	Example / Usage
Program Counter (PC)	Holds the address of the next instruction to be fetched	PC = 0x0040 → fetch instruction from address 0x0040
Instruction Register (IR)	Holds the currently executing instruction	IR = MOV R1, R2 → decoded and executed by control unit
Memory Address Register (MAR)	Holds the address of the memory location to be accessed (read/write)	MAR = 0x2000 → access memory at address 0x2000
Memory Data Register (MDR) / Memory Buffer Register (MBR)	Temporarily holds data being transferred to/from memory	MDR = 42 → data to be written to memory at MAR address
Status Register / PSW (Processor Status Word)	Holds flags indicating the outcome of operations and control bits	PSW: Z=1 (zero), C=0 (no carry), N=0, V=0 → after comparison
Control Registers	Used to control processor operations like interrupts, modes, etc.	Interrupt Enable = 1 → allows interrupt handling

Design Issues

1. Number of Registers:

- A higher number of registers reduces memory access frequency, improving performance.
- However, it increases hardware complexity and the overhead during context switches.
- RISC architectures (e.g., MIPS, ARM) generally have more (e.g., 32) general-purpose registers. CISC architectures (e.g., x86) often use fewer, specialized registers.

Contd...(design issues)

2. Types and Functions of Registers

- Registers can be general-purpose or special-purpose.
- General-purpose registers (GPRs) are used for arithmetic, logic, and data handling.
- Special-purpose registers (e.g., PC, IR, SP) have fixed roles in instruction execution.
- Design choice affects instruction set flexibility and hardware control logic.

Contd...

3. Register Addressing and Access Modes

- Direct access: Registers explicitly named in instructions (e.g., MOV R1, R2).
- Indirect/Implicit access: Some architectures use accumulator or stack-based access.
- Efficient register access simplifies decoding and improves execution speed

4. Register Length:

- Large enough to hold the largest address
- Able to hold values of most data types

...

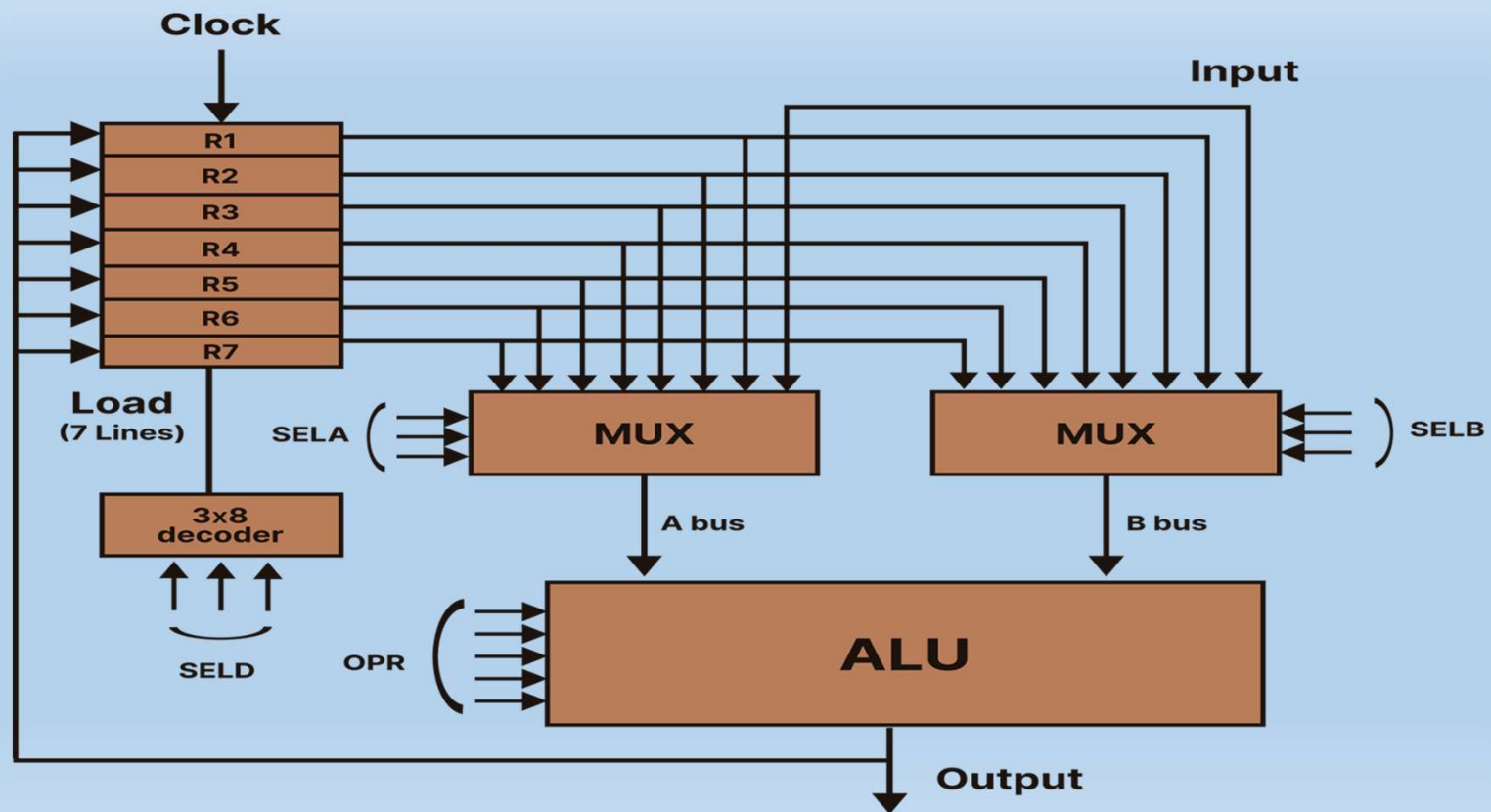
5. Instruction Set Design and Register Use

- Affects register pressure:
 - 3-address instruction (e.g., ADD R1, R2, R3) is more flexible.
 - Load/store architecture separates memory and register operations.
- RISC favors register-centric operations; CISC allows memory operands.

General Register Organization

- The system described is a CPU bus organization that uses seven registers connected to two multiplexers (MUX), forming two buses, A and B. These buses connect to an Arithmetic Logic Unit (ALU), which performs various arithmetic or logic operations based on control signals. The result is then routed to the output bus, which feeds back into the registers, with one selected register receiving the result.

General Register Organization Implementation Block



Components of General Register Organization

Registers and Buses

- Each register is connected to two multiplexers (MUX).
- The output of these registers is sent to buses A and B through the multiplexers.
- The MUX selection lines determine which register data is placed onto each bus.

ALU (Arithmetic Logic Unit)

- Buses A and B feed the ALU, which performs operations like addition or subtraction depending on the control signal.
- The ALU's output is then sent to the output bus.

Contd...

Register Load

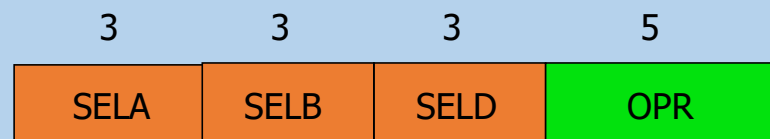
- A decoder controls which register will receive the ALU's result from the output bus.
- The destination register is selected via the decoder, which activates the load input of the chosen register.

Contd...

Control Unit

- The control unit generates four key control signals:
- **MUX A selector (SELA):** Chooses the source register for bus A.
- **MUX B selector (SELB):** Chooses the source register for bus B.
- **ALU operation selector (OPR):** Defines the ALU operation (e.g., addition).
- **Decoder destination selector (SELD):** Chooses which register will load the result.

Register set with common ALU(*cont.*)



(b) Control word

Example

For the operation $R1 \leftarrow R2 + R3$, the control signals would:

- Set SELA to select R2 for bus A.
- Set SELB to select R3 for bus B.
- Set OPR to perform addition in the ALU.
- Set SELD to select R1 as the destination register.

Encoding of register selection fields

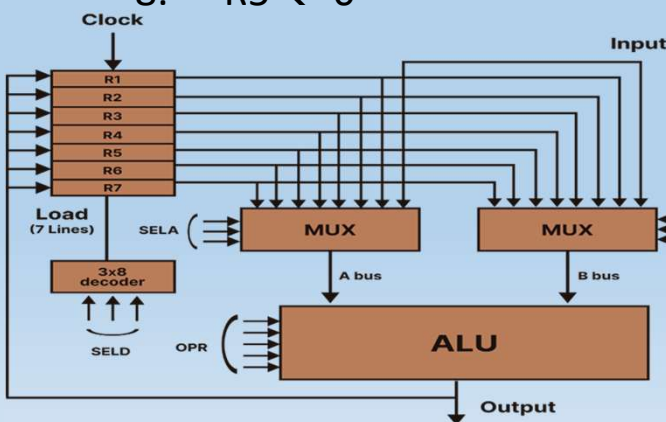
Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Encoding of ALU operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add A + B	ADD
00101	Subtract A – B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Write control word in symbolic and binary form for the following operations

1. $R1 \leftarrow R2 - R3$
2. $R4 \leftarrow R4 \vee R5$
3. $R6 \leftarrow R6 + 1$
4. $R7 \leftarrow R1$
5. $\text{Output} \leftarrow R2$
6. $\text{Output} \leftarrow \text{Input}$
7. $R4 \leftarrow \text{shl } R4$
8. $R5 \leftarrow 0$



Microoperation	Symbolic designation				Control word
	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

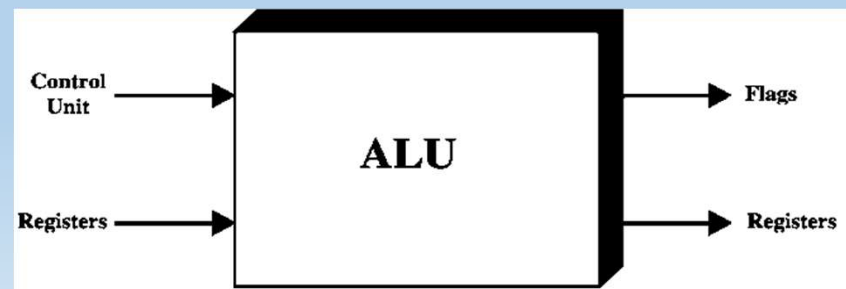
OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Examples of microoperations for the CPU

Microoperation	Symbolic designation				Control word			
	SELA	SELB	SELD	OPR				
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010	011	001	00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100	101	100	01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110	000	110	00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001	000	111	00000
$\text{Output} \leftarrow R2$	R2	—	None	TSFA	010	000	000	00000
$\text{Output} \leftarrow \text{Input}$	Input	—	None	TSFA	000	000	000	00000
$R4 \leftarrow \text{shl } R4$	R4	—	R4	SHLA	100	000	100	11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101	101	101	01100

Arithmetic and Logic Unit

- ALU is the combinational circuit of that part of computer that actually performs arithmetic and logical operations on data.
- All of the other elements of computer system- control unit, registers, memory, I/O are mainly to bring data into the ALU for it to process and then to take the result back out.
- An ALU & indeed all electronic components in computer are based on the use of simple digital logic device that can store binary digit and perform simple Boolean logic function. Figure indicates in general in general term how ALU is interconnected with rest of the processor.



ALU(contd...)

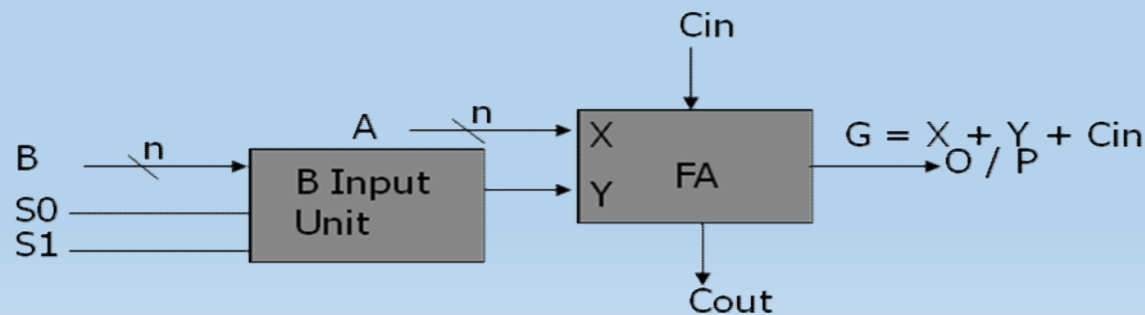
- Data are presented to ALU in register and the result of operation is stored in register. These registers are temporarily storage location within the processor that are connected by signal path to the ALU. The ALU may also set flags as the result of an operation.
- The flags values are also stored in registers within the processor. The control unit provides signals that control the operation of ALU and the movement of data into and out of ALU.

Stages of ALU

- Design Arithmetic Unit
- Design Logical Unit
- Combine Both and form the single unit

Stage 1: Design the arithmetic section

- The basic component of arithmetic circuit is a parallel adder which is constructed with a number of full adder circuits connected in cascade.
- By controlling the data inputs to the parallel adder, it is possible to obtain different types of arithmetic operations. Below figure shows the arithmetic circuit.

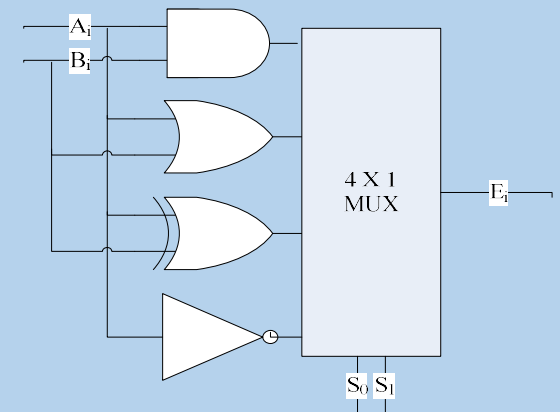


Functional Table of Arithmetic Circuit

Select		Input Y	Output		Microoperation	
S_1	S_0		Cin = 0	Cin = 1	Cin = 0	Cin = 1
0	0	0	A	A+1	Transfer A	Increment A
0	1	B	A+B	A+B+1	Addition	Addition with carry
1	0	B'	A+B'	A+B'+1	Subtraction with borrow	Subtraction
1	1	-1	A-1	A	Decrement A	Transfer A

Stage 2: Design The logical Section

- The basic components of logical circuit are AND, OR, XOR and NOT gate circuits connected accordingly.
- Below figure shows a circuit that generates four basic logic micro-operations. It consists of four gates and a multiplexer. Each of four logic operations is generated through a gate that performs the required logic. The two selection input S1 and S0 choose one of the data inputs of the multiplexer and directs its value to the output.

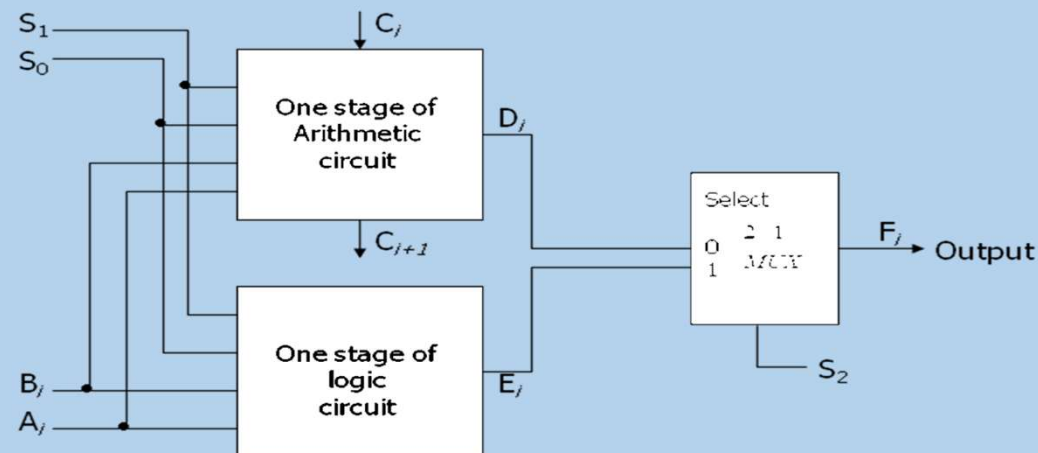


Functional Table for Logical Section

S1	S0	Output	Microoperation
0	0	$A_i \&\& B_i$	AND
0	1	$A_i \mid\mid B_i$	OR
1	0	$A_i \text{ XOR } B_i$	XOR
1	1	A_i'	NOT

Combine these 2 sections to form the ALU

- Below figure shows a combined circuit of ALU where n data input from A are combined with n data input from B to generate the result of an operation at the G output line.



Contd...

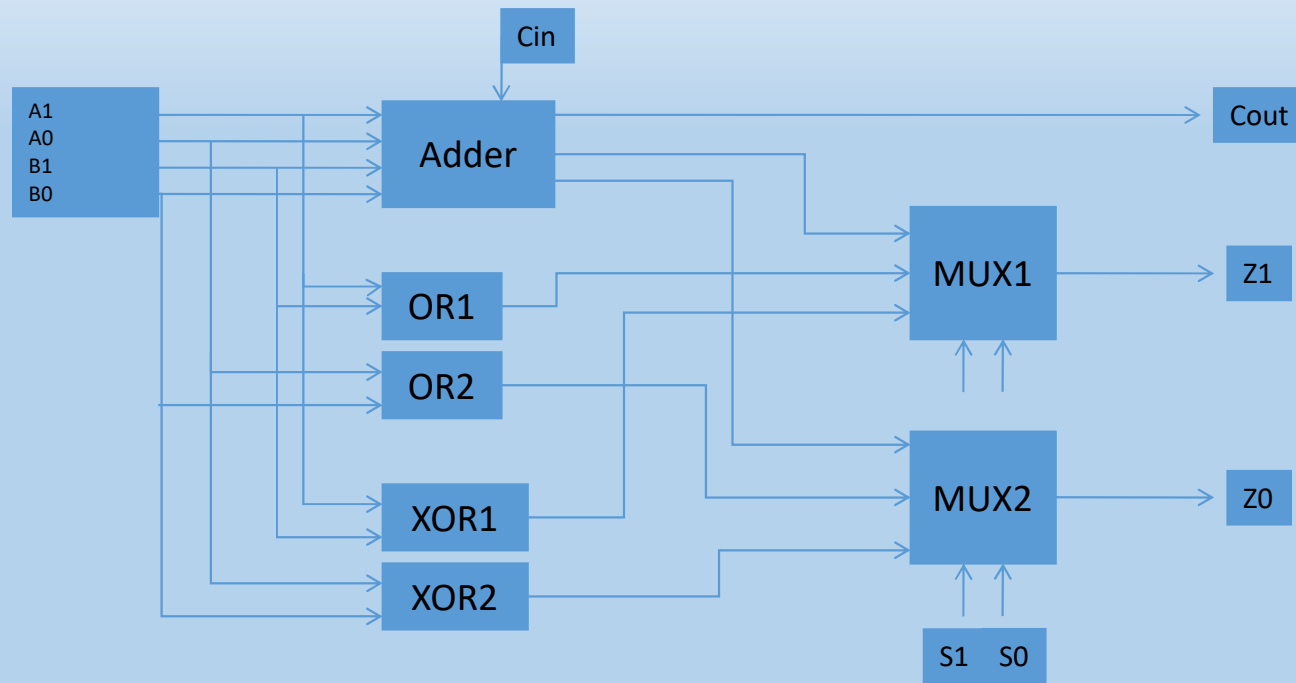
- ALU has a number of selection lines used to determine the operation to be performed. The selection lines are decoded with the ALU so that selection lines can specify distinct operations.
- The mode select S_2 differentiate between arithmetic and logical operations. The two functions select S_1 and S_0 specify the particular arithmetic and logic operations to be performed.
- With three selection lines, it is possible to specify arithmetic operation with S_2 at 0 and logical operation with S_2 at 1.

Functional Table of ALU

S_2	S_1	S_0	Cin	Input Y	Operation Type	Output / Result	Micro-operation
0	0	0	0	0	Arithmetic	A	Transfer A
0	0	0	1	0	Arithmetic	$A + 1$	Increment A
0	0	1	0	B	Arithmetic	$A + B$	Addition
0	0	1	1	B	Arithmetic	$A + B + 1$	Addition with carry
0	1	0	0	B'	Arithmetic	$A + B'$	Subtraction with borrow
0	1	0	1	B'	Arithmetic	$A + B' + 1$	Subtraction
0	1	1	0	-1	Arithmetic	$A - 1$	Decrement A
0	1	1	1	-1	Arithmetic	A	Transfer A
1	0	0	X	B	Logical	$A \text{ AND } B$	Logical AND
1	0	1	X	B	Logical	$A \text{ OR } B$	Logical OR
1	1	0	X	B	Logical	$A \text{ XOR } B$	Logical XOR
1	1	1	X	B	Logical	NOT A	Logical NOT

Example

- Design 2-bit ALU performing addition, OR and XOR



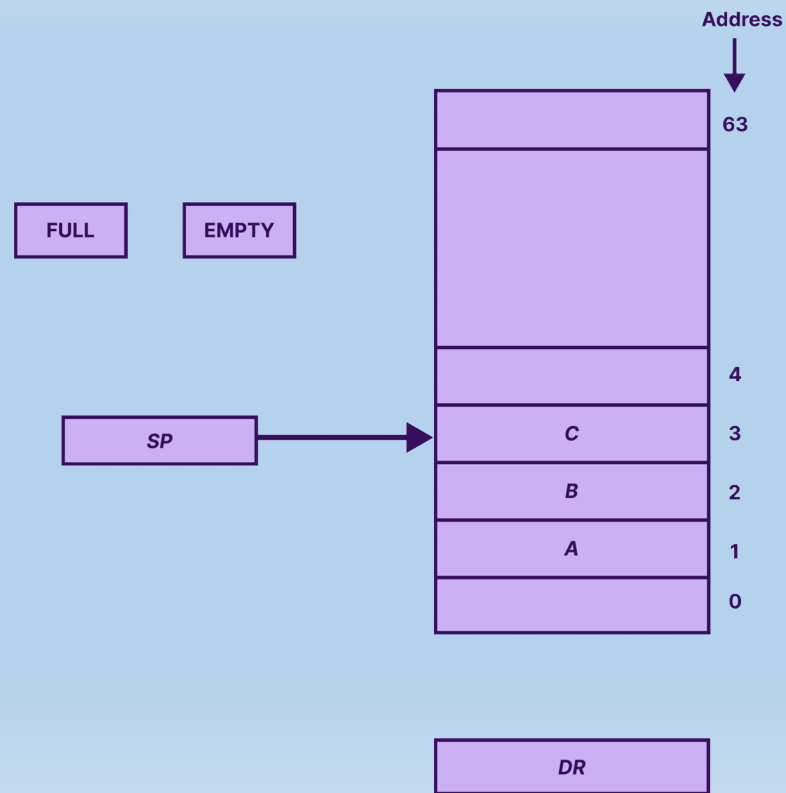
Stack Organization

- A stack is a data storage structure in which the most recent thing deposited is the most recent item retrieved.
- It is based on the LIFO concept (Last-in-first-out). The stack is a collection of memory locations containing a register that stores the top-of-element address in digital computers.
- Stack's operations are:
 - Push: Adds an item to the top of the stack
 - Pop: Removes one item from the stack's top

Stack Organization

- Stack organization refers to the way a computer system uses a stack for managing execution flow, storing temporary data, and performing arithmetic or logical operations. The stack is implemented as a portion of memory, with a stack pointer (SP) that tracks the current top of the stack.
- In stack organization, when an operation is performed, the stack pointer is updated. This makes stack operations efficient and allows for orderly memory access.

Stack Organization



64 Word Stack

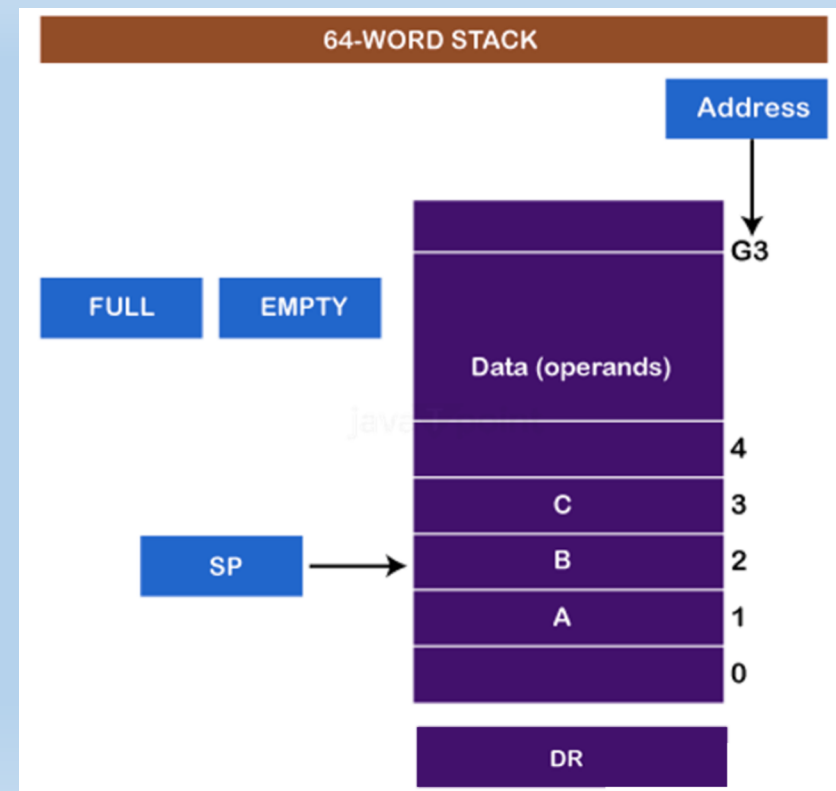
Types of Stack Organization

- There are two types of stack organization which are used in the computer hardware.
 - Register stack- It is built using register
 - Memory stack- It is a logical part of memory allocated as stack.

Register Stack

- Consider a 64-word register stack like the one shown in the diagram.
- A binary number, which is the address of the element at the top of the stack, is stored in the stack pointer register.
- The stack has the three elements A, B, and C.

SP (Stack Pointer): Holds the address of the top of the stack
DR (Data Register): Holds data written to or read from memory
FULL: Indicates stack overflow
EMPTY: Indicates stack underflow (empty stack)



Stack Organization (Operation)

- The stack pointer holds C's address, which is 3. C is at the top of the stack.
- The top element is removed from the stack by reading the memory word at address 3 and decreasing the stack pointer by one.
- As a result, B is at the top of the stack, and SP now contains the address of B (2).
- It may add a new word to the stack by increasing the stack pointer and inserting a word in the newly increased location.

Contd...

- Because $2^6 = 64$ and the SP cannot exceed 63, the stack pointer has 6 bits (111111 in binary). Incrementing 63 by 1 causes overflow, and only the lower 6 bits are stored, resulting in 0 (i.e. $111111 + 1 = 1000000$). Only the six least important bits are stored in SP. The result of decrementing 000000 by one is 111111.
- As a result, the one-bit register 'FULL' is set to 1 when the stack is full. The binary information constructed into or readout of the stack is stored in the data register DR.

Contd...

- First, the SP is set to 0, the EMTY to 1, and the FULL to 0.
- The push operation is used to insert a new piece because the stack is not yet full (FULL = 0).

Push Operation:

If (SP = 63) then FULL $\leftarrow$ 1 // stack overflow check

Else

SP $\leftarrow$ SP + 1 // increment stack pointer

M[SP] $\leftarrow$ DR // store data in memory

EMTY $\leftarrow$ 0

POP Operation

First, the stack pointer (SP) points to the top element of the stack, the **EMPTY flag is 0**, and the **FULL flag is 0**. The **pop operation** is performed to remove the top element because the stack is **not empty (EMPTY = 0)**. During the pop operation, the **memory word addressed by SP is read into the data register (DR)**, and then the **stack pointer is decremented by one**.

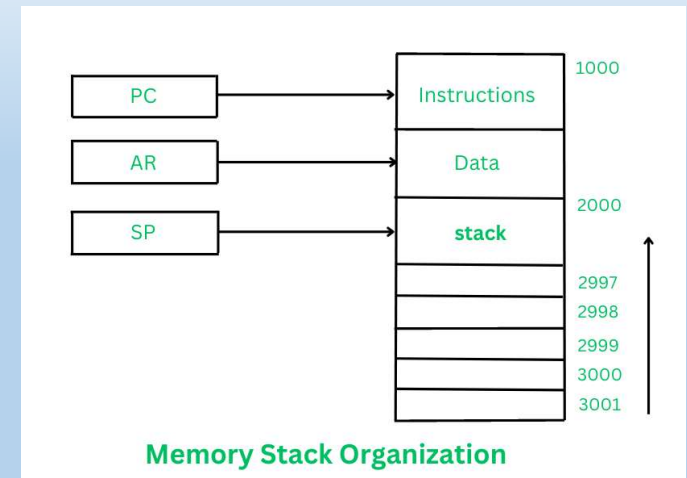
As a result, the **previous element becomes the new top of the stack**.

If the stack pointer reaches **-1**, the stack becomes empty and the **EMPTY flag is set to 1**.

```
If (SP = -1) then
    EMPTY ← 1    // stack underflow
Else
    DR ← M[SP]   // read top element
    SP ← SP - 1  // decrement stack pointer
    FULL ← 0     // stack is no longer full
```

Memory Stack Organization

- A portion of memory is assigned to a stack operation to implement the stack in the CPU.
- Here the processor register is used as a Stack Pointer (SP).
- The portion of computer memory divided into three segments: Program Instructions, Data, and Stack.



- **Program Counter (PC):** It is a register that points to the address of the **next instruction** that is going to be executed in the program.
- **Address Register (AR):** This register points at the collection of data and is used during the execute phase to **read an operand**.
- **Stack Pointer (SP):** It points at the top of the stack and is used to **push or pop** the data items in or from the stack.

Descending Stack (Memory Stack)

- Stack Pointer Initialization:
 - SP points to the top of the stack initially: SP = 3001.
- Stack Growth:
 - The stack grows downwards, i.e., toward lower memory addresses.
 - First element → stored at 3001
 - Second element → stored at 3000
 - ... continues until the last allowed address 2000.
- Data Transfer
 - Data to be pushed comes from the Data Register (DR).
 - Data popped from the stack is also read into the Data Register (DR)

PUSH

- This operation is used to insert a new data item into the top of the Stack. The new item can be inserted as follows:-

$$SP \leftarrow SP-1$$

$$M[SP] \leftarrow DR$$

- In the first step, the Stack Pointer is **decremented** to point at the address where the data item will be stored.
- Then, by using the memory write operation, the data item from Data Register gets inserted into the top of the stack (at the address where the Stack Pointer is pointing).

POP

- This operation is used to delete a data item from the top of the Stack. Data item can be deleted as follows:-
 - $DR \leftarrow M[SP]$; read data from stack into DR
 - $SP \leftarrow SP + 1$; move SP to point to the top element
- In the first step, the top data item is read from the Stack into the Data Register.
- The Stack Pointer is then **incremented** to point at the next data item in the stack. Push or Pop operations can be performed with the help of the following microoperations:
 - Access to memory with the help of Stack Pointer (SP), and
 - Updating the stack.

Reverse Polish Notation In Stack

- The reverse polish notation in the stack is also known as postfix expression. Here, we use stack to solve the postfix expression.
- Operators come **after operands**.
- From the postfix expression, when some operand is found, we push it into the stack, and when some operator is found, we pop elements from the stack, and after that, the operation is performed in the correct sequence, and the result is also stored in the stack.

Contd...

- For example, we are given this expression in the form of an array,

```
["18", "5", "*", "6", "/"]
```

From the given array we can deduce expression as,
 $((18 * 5) / 6) = 15$

Evaluating Arithmetic Expressions

- Stack-based architectures are particularly efficient for evaluating infix expressions converted into postfix (Reverse Polish) notation.

- Expression:**

$(5 + 3) * (8 - 2)$

- Postfix:**

$5\ 3\ +\ 8\ 2\ -\ *$

Execution Steps		
Step	Instruction	Stack Content (Top on Right)
1	PUSH 5	5
2	PUSH 3	5 3
3	ADD	8
4	PUSH 8	8 8
5	PUSH 2	8 8 2
6	SUB	8 6
7	MUL	48

Instruction Formats

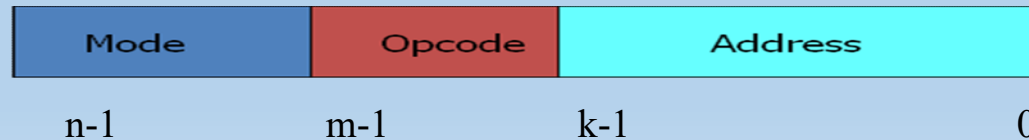
- The computer can be used to perform a specific task, only by specifying the necessary steps to complete the task. The collection of such ordered steps forms a 'program' of a computer.
- These ordered steps are the instructions. Computer instructions are stored in central memory locations and are executed sequentially one at a time. The control reads an instruction from a specific address in memory and executes it. It then continues by reading the next instruction in sequence and executes it until the completion of the program.

Instruction Formats

- A computer usually has a variety of Instruction Code Formats. It is the function of the control unit within the CPU to interpret each instruction code and provide the necessary control functions needed to process the instruction.
- An n bit instruction that k bits in the address field and m bits in the operation code field come addressed 2^k location directly and specify 2^m different operation.
- The bits of the instruction are divided into groups called fields.

Contd...

- The most common fields found in instruction formats are:
 - An operation code field that specifies the operation to be performed.
 - An address field that designates a memory address or a processor registers.
 - A mode field that specifies the way the operand or the effective address is determined.



Contd...

- The operation code field (Opcode) of an instruction is a group of bits that define various processor operations such as add, subtract, complement, shift etc.
- The bits that define the mode field of an instruction code specify a variety of alternatives for choosing the operands from the given address.
- Operation specified by an instruction is executed on some data stored in the processor register or in the memory location. Operands residing in memory are specified by their memory address. Operands residing in processor register are specified with a register address.

Types of Instruction Format

- **Based on the number of addresses, instructions are**
 - Three – Address Instructions
 - Two – Address Instructions
 - One – Address Instructions
 - Zero – Address instructions
- **Based on internal organization**
 - Single Accumulator Organization
 - General Register Organization
 - Stack Organization

...

- **Three – Address Instructions**

- Each address field is used to specify either a processor register or a memory operand
- Its advantage is that the program becomes shorter while evaluating arithmetic and logic expressions
- The disadvantage is that the instructions require too many bits to specify three addresses.
- To add two numbers: `ADD R1, R2, R3`

...

- **Two – Address Instructions**

- The address field can specify either a processor register or a memory word.
- Compared to three address instructions, programs, with two address instructions, are longer and fewer bits are required to specify only two addresses.
- MOV instruction is used
- To add two numbers: ADD R1, R2 and ADD R1, R3

...

- **One – Address Instructions**

- implied accumulator register for all data manipulation
- all operations are done between the accumulator register and a memory operand
- accumulator contains the result of all operations
- the number of bits representing the instruction is reduced
- the program for given arithmetic expression will be longer as compare to three or two address instructions
- LOAD/STORE instructions are used
- To add two number: LDA X, ADD Y, STA Z

...

- **Zero – Address instructions**

- does not use an address field for some instructions, such as ADD and MUL
- the PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack
- For any given operation, the instruction gets longer compared with other formats
- Operation in stack will be slower compared to registers
- To add two numbers: PUSH X, PUSH Y, ADD, POP Z

Design Issues of Instruction Format

- **Instruction Length**

- more opcodes, more operands, more addressing modes, and greater address range to make programming easier
- shorter programs can be written to accomplish given tasks.
- require more bits and results in longer instruction lengths
- instruction length to be equal to the memory-transfer length (in a bus system, data-bus length) or one should be a multiple of the other

- **Allocation of bits**

- more opcodes means few bits for addressing

Variable Instruction Formats

- Provides a large repertoire of opcodes, with different opcode lengths.
- Addressing can be more flexible, with various combinations of register and memory references plus addressing modes.
- Increases complexity of the processor
 - Usually, the processor does not know the length of the next instruction to be fetched
 - the processor can fetch a number of bytes or words equal to the at least the longest possible instruction
 - this results in multiple instructions being fetched

ADDRESSING MODES

- Defines a mechanism to fetch the content of an operand
- It facilitates programmer with
 - Pointers to memory
 - Counters for loop control
 - Indexing of data



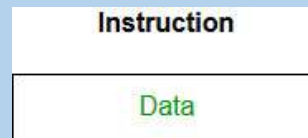
Format of Instruction

Types of Addressing Modes

- Implied Addressing Mode
- Immediate Addressing Mode
- Direct Addressing Mode
- Indirect Addressing Mode
- Register Direct Addressing Mode
- Register Indirect Addressing Mode
- Displacement Addressing Mode
 - Relative Addressing
 - Base-Register Addressing
 - Indexed Addressing

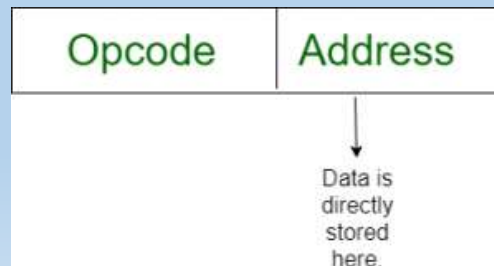
Implied Addressing Mode

- Operands defined implicitly in the instruction
- Example: CMA, XCHG (in 8085)



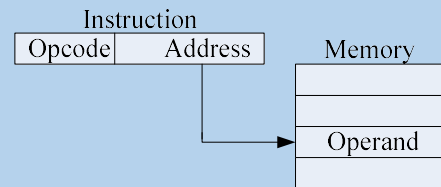
Immediate Addressing Mode

- operand value present in instruction.
- used to define and use constants or set initial values of variables.
- The data will be stored in twos complement form; the left most bit of the operand field is used as a sign bit.
- Example: MVI B, 55H (in 8085)



Direct Addressing Mode

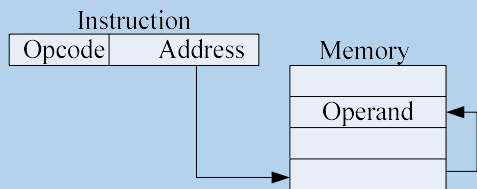
- address field contains the effective address of the operand
- requires only one memory reference and no special calculation
- Example: STA 9050H (in 8085)



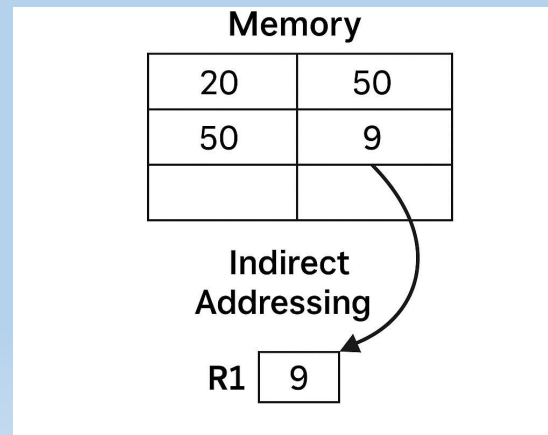
Effective Address (EA) = A

Indirect Addressing Mode

- limited address range can be resolved using this addressing
- address field refer to the address of a word in memory, which in turn contains a full-length address of the operand.
- Example: MOV R1, @20 (in 8051)

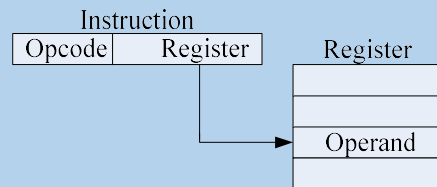


Effective Address (EA) = (A)



Register Direct Addressing Mode

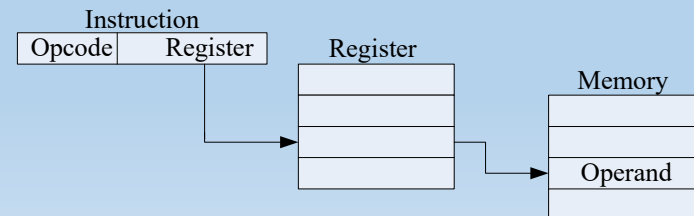
- In this mode, the address field represents the register as the intended address and the operand value is contained in that particular register.
- Example: MOV A, B (in 8085)



Effective Address (EA) = R

Register Indirect Addressing Mode

- Instruction specifies a register in the processor whose contents give the address of the operand in memory.
- Memory address of the operand is needed to be placed in the registers in advance
- Example: STAX B (in 8085)



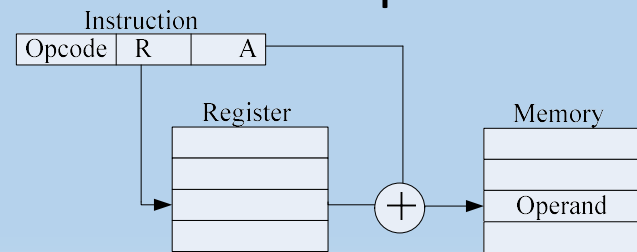
Effective Address (EA) = (R)

Displacement Addressing Mode

- It combines the capabilities of direct addressing and register indirect addressing.
- It requires the instruction to have two address fields, at least one of which is explicit.
- An implicit reference, based on opcode, refers to a register whose contents are added to the value of address field to produce the effective address.

Instruction: `add r4,100 (r1)`

Operation: `r4 <- r4 + M[100+r1]`



$$\text{Effective Address (EA)} = A + (R)$$

...

- **Relative Addressing**

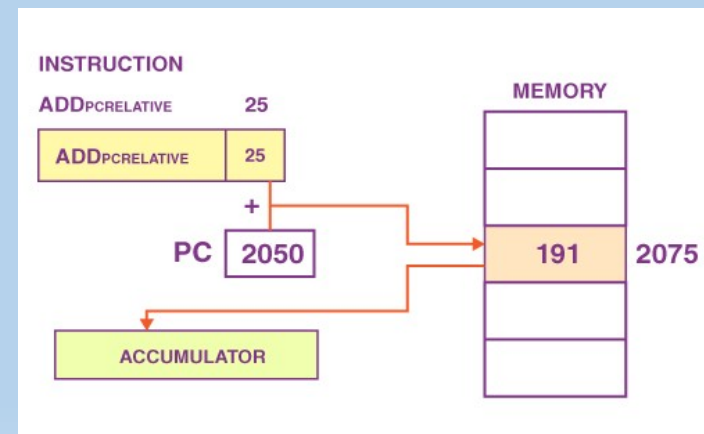
- The program counter (PC) is implicitly referenced; the next instruction address is added to the address field to produce the effective address

Here is a symbolic representation of the relative address mode:

$X(PC)$

Here is how the effective address for it:

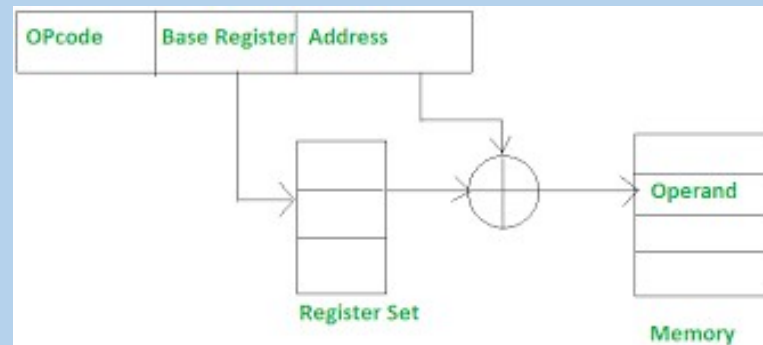
$$EA = X + (PC)$$



...

- **Base-Register Addressing**

- the content of a base register is added to the address part of the instruction to obtain the effective address.
- A base register is assumed to hold a base address and the address field of the instruction gives a displacement relative to this base address.

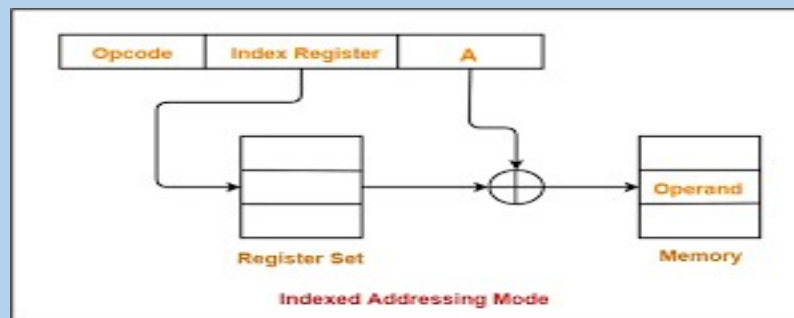


Base register addressing, a form of displacement addressing, is defined by the equation $EA = A + (R)$, where A represents the displacement and R holds the pointer to the base address.

...

- **Indexed Addressing**

- The content of an index register is added to the address part of the instruction to obtain the effective address



$$EA = A + (R)$$

Pre-indexing ($EA = A + (R++)$) and post-indexing ($EA = A + ({}++R)$)

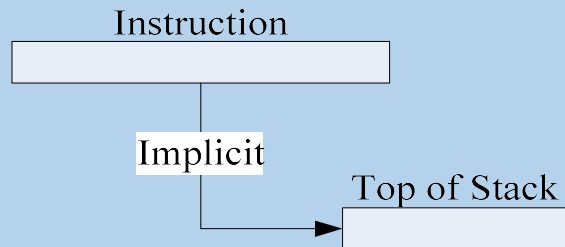
Auto increment and decrement Mode

- Similar to the register indirect mode except that
 - »the register is incremented after its value is used to access memory
 - »the register is decrement before its value is used to access memory
 - »Example (Auto increment) : LD (R1)+

$$AC \leftarrow M[R1], R1 \leftarrow R1 + 1$$

Stack Addressing mode

- The stack is the linear array of locations. It is some times referred to as push down list or last in First out (LIFO) queue. The stack pointer is maintained in register.



Effective Address (EA) = TOS

Illustrative Example on Addressing Mode

Addressing Mode	EA	AC content
Direct Address	500	800
Immediate Address	201	500
Indirect Address	800	300
Relative address	702 (202+500 as PC=202 and A=500)	325
Indexed Address	500+100=600 as XR+A	900
Register Direct	-	400
Register Indirect	400	700
Auto decrement	399	450
Auto Increment	400	700

PC = 200
R1 = 400
XR = 100
AC

Address	Memory
200	Load to AC Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

Data Transfer and Manipulation

- Computers provide an extensive set of instructions to give the user the flexibility to carry out various computational tasks.
- Most computer instructions can be classified as data transfer, data manipulation and program control instructions.

Data Transfer Instructions

- move data from one place to another without changing its content
- common transfers
 - between memory and processor registers
 - between processor registers and input or output
 - between the processor register themselves
- Examples: load, store, move, input, output, push, pop etc.

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

Data Manipulation Instructions

- perform operations on data and provide the computational capabilities for the computer
- The data manipulation instructions can be further divided as
 - **Arithmetic instructions**
 - **Logical and Bit Manipulation Instructions**
 - **Shift Instructions**

Arithmetic instructions

- basic arithmetic operations are addition, subtraction, multiplication, and division
- operations may be available for different types of data
- special carry flip-flop is used to store the carry from an operation.
- Examples: INC, DEC, ADD, SUB, MUL, DIV

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

Logical and Bit Manipulation Instructions

- It performs binary operations on strings of bits stored in registers
- It is useful in manipulating individual bits or a group of bits that represent binary-coded information
- It considers each bit of the operand separately and it is possible to change bit value, clear a group of bits or insert new bit value into operands.
- Examples: CLR, SET, XOR, COM, AND, OR, XOR

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

APPLICATIONS OF LOGIC MICROOPERATIONS

- Logic microoperations can be used to manipulate individual bits or a portions of a word in a register
- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A
 - Selective-set $A \leftarrow A + B$
 - Selective-complement $A \leftarrow A \oplus B$
 - Selective-clear $A \leftarrow A \cdot B'$
 - Mask (Delete) $A \leftarrow A \cdot B$
 - Clear $A \leftarrow A \oplus B$
 - Insert $A \leftarrow (A \cdot B) + C$
 - Compare $A \leftarrow A \oplus B$

SELECTIVE SET

- In a selective set operation, the bit pattern in B is used to *set* certain bits in A

$$\begin{array}{r}
 1\ 1\ 0\ 0A_t \\
 1\ 0\ 1\ 0B \\
 \hline
 1\ 1\ 1\ 0A_{t+1}
 \end{array}
 \quad (A \leftarrow A + B)$$

- If a bit in B is set to 1, that same position in A gets set to 1, otherwise that bit in A keeps its previous value

SELECTIVE COMPLEMENT

- In a selective complement operation, the bit pattern in B is used to *complement* certain bits in A

$$\begin{array}{r}
 1\ 1\ 0\ 0\ A_t \\
 1\ 0\ 1\ 0\ B \\
 \hline
 0\ 1\ 1\ 0\ A_{t+1}
 \end{array}
 \quad (A \leftarrow A \oplus B)$$

- If a bit in B is set to 1, that same position in A gets complemented from its original value, otherwise it is unchanged

SELECTIVE CLEAR

- In a selective clear operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{rcl}
 1\ 1\ 0\ 0 & A_t & \\
 1\ 0\ 1\ 0 & B & \\
 \hline
 1\ 0\ 0\ 0 & A_{t+1} & (A \leftarrow A \cdot B')
 \end{array}$$

- If a bit in B is set to 1, that same position in A gets set to 0, otherwise it is unchanged

MASK OPERATION

- In a mask operation, the bit pattern in B is used to *clear* certain bits in A

$$\begin{array}{r}
 1\ 1\ 0\ 0A_t \\
 1\ 0\ 1\ 0B \\
 \hline
 1\ 0\ 0\ 0A_{t+1}
 \end{array}
 \quad (A \leftarrow A \cdot B)$$

- If a bit in B is set to 0, that same position in A gets set to 0, otherwise it is unchanged

CLEAR OPERATION

- In a clear operation, if the bits in the same position in A and B are the same, they are cleared in A, otherwise they are set in A

$$\begin{array}{rcl}
 1\ 1\ 0\ 0 & A_t & \\
 1\ 0\ 1\ 0 & B & \\
 \hline
 0\ 1\ 1\ 0 & A_{t+1} & (A \leftarrow A \oplus B)
 \end{array}$$

INSERT OPERATION

- An insert operation is used to introduce a specific bit pattern into A register, leaving the other bit positions unchanged
- This is done as
 - A mask operation to clear the desired bit positions, followed by
 - An OR operation to introduce the new bits into the desired positions
 - Example
 - Suppose you wanted to introduce 1010 into the low order four bits of A:

1101 1000 1011 0001 A (Original)

1101 1000 1011 1010 A (Desired)

- | | |
|---------------------|------------------|
| 1101 1000 1011 0001 | A (Original) |
| 1111 1111 1111 0000 | Mask |
| 1101 1000 1011 0000 | A (Intermediate) |
| 0000 0000 0000 1010 | Added bits |
| 1101 1000 1011 1010 | A (Desired) |

Shift Instructions

- shift the bits of a word to the left or right
- may specify logical shift, arithmetic shift or rotate type operations
- logical shift inserts zero to the end bit position
- arithmetic shift usually conform to the rules for signed two's complement numbers
- arithmetic shift right instruction must preserve the sign bit of in the leftmost position
- arithmetic shift left instruction inserts zero to the end position and is identical to the logical shift left instruction.

...

- In rotate instructions, bits shifted out at one end of the word are not lost as in a logical shift but are circulated back into the other end
- The rotate through carry instruction treats a carry bit an extension of the register whose word is being rotated
- Examples: SHR, SHL, RAL, RAR, RLC, RRC

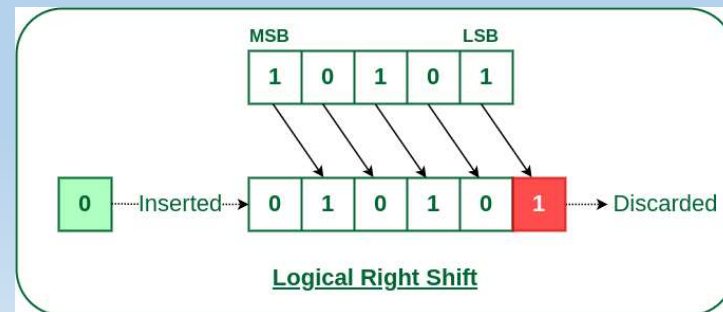
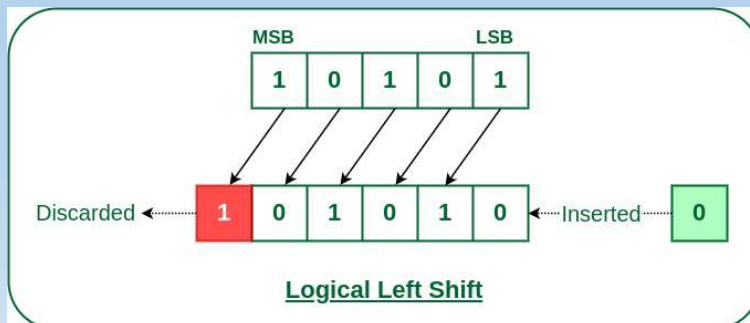
Shifter

- Logical shift: A logical shift is one that transfers 0 through the serial input. We will adopt the symbols *shl* and *shr* for logical shift-left and shift-right micro operations.

$R1 \leftarrow \text{shl } R1$

$R2 \leftarrow \text{shr } R2$

The bit transferred to the end position through the serial input is assumed to be 0 during a logical shift.

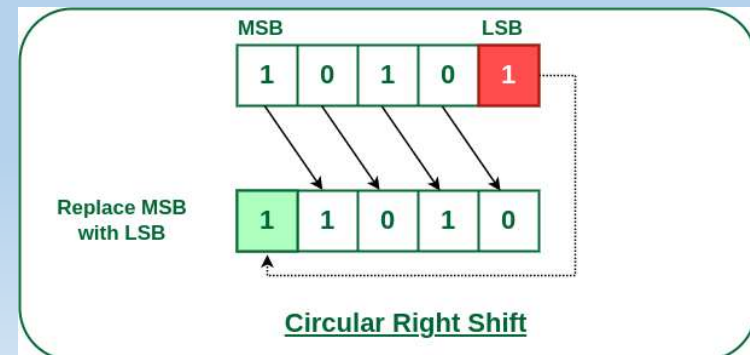
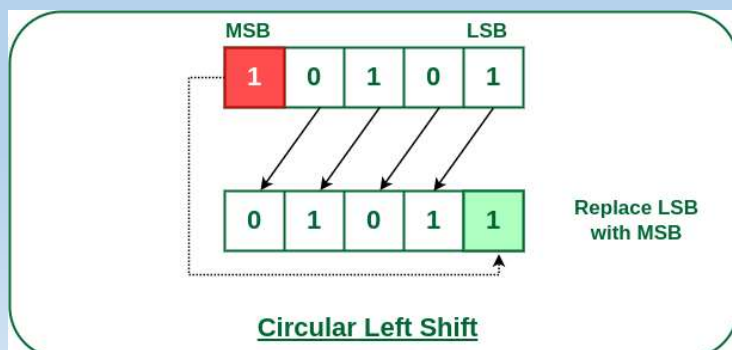


Circular Shift

Circular Shift: The circular shift(also known as a rotate operation) circulates the bits of the register around the two ends without loss of information.

This is accomplished by connecting the serial output of the shift register to its serial input.

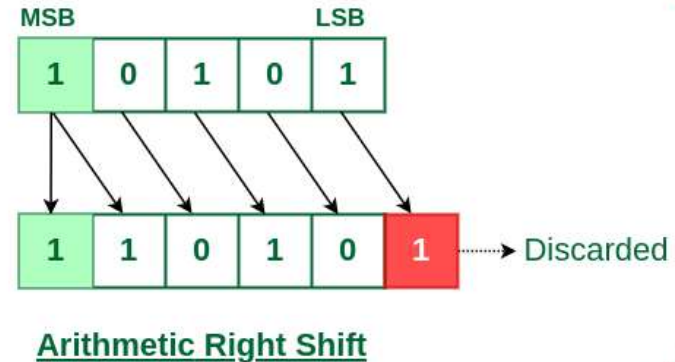
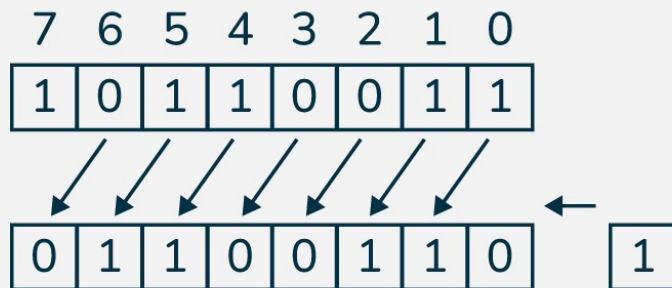
We will use the symbols *CIL* and *CIR* for the circular shift left and right, respectively.



Arithmetic Shift:

- An arithmetic shift is a micro-operation that shifts a signed binary number to the left or right.
- An arithmetic shift-left multiplies a signed binary number by 2. o An arithmetic shift-right divides the number by 2.
- Arithmetic shifts must leave the sign bit unchanged because the sign of the number remains the same when it is multiplied or divided by 2.

Shift Left Arithmetic



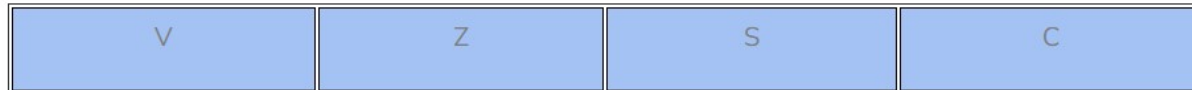
Program Control Instruction

- The program control instructions provide decision making capabilities and change the path taken by the program when executed in computer.
- These instructions specify conditions for altering the content of the program counter.
- The change in value of program counter as a result of execution of program control instruction causes a break in sequence of instruction execution.

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

Status Bit Conditions

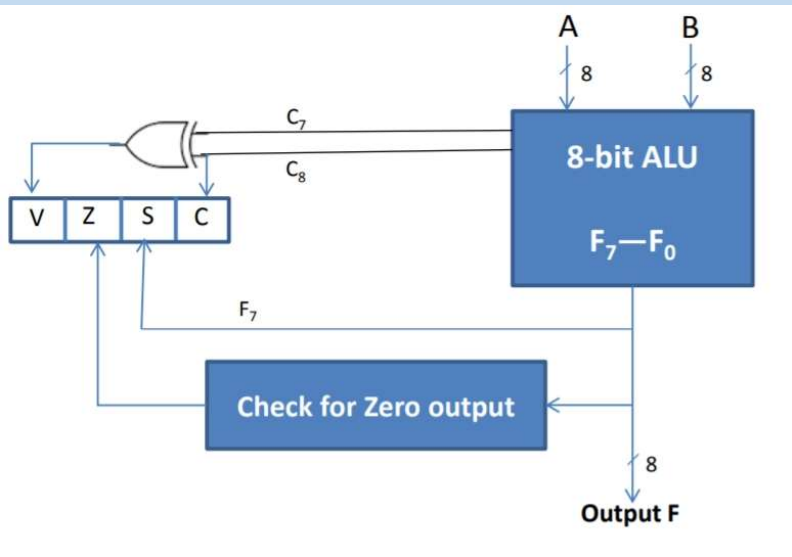
- To check different conditions for branching instructions like CMP (compare) or TEST can be used. Certain status bit conditions are set as a result of these operations.
- Status bits mean that the value will be either 0 or 1 as it is a bit. We have four status bits:
 - "V" stands for Overflow
 - "Z" stands for Zero
 - "S" stands for the Sign bit
 - "C" stands for Carry.



Contd...

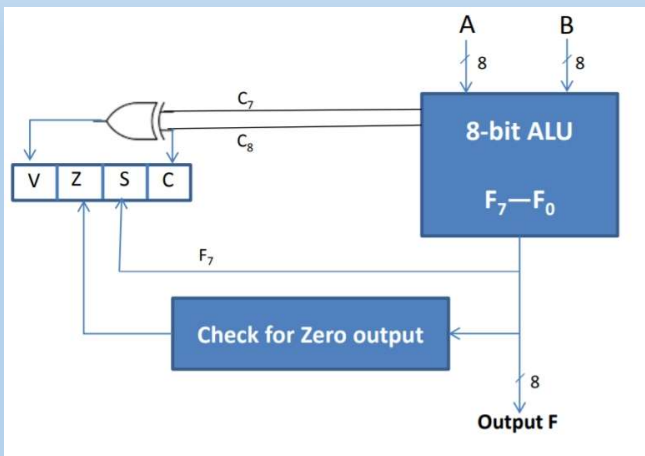
- Now, these will be set or reset based on the ALU (Arithmetic Logic Unit) operation carried out into the CPU. Let us discuss these bits before understanding the operation.
 - Overflow(V) is based on certain bits, i.e., if extra bits are generated into our operation.
 - Then we have Zero (Z), if the output of the ALU(Arithmetic Logic Unit) is 0, then the Z flag is set to 1, otherwise, it is set to 0.
 - If the number is positive, the Sign(S) flag is 0, and if the number is negative, the Sign flag is 1.
 - We have Carry(C), if the output of the ALU operation generates Carry, then C is set to 1, else C is set to 0.

Status Bit condition



- The figure shown alongside is an 8-bit ALU that performs arithmetic or logic operations on our data.
- Suppose we have two operands A and B of 8-bits on which we are performing certain arithmetic or logic operations.
- At current instant, addition is performed on A and B. We know that an extra Carry bit may be generated, which means eight Carry bits are generated.
- If the addition operation is performed on operands A and B, then the carry bits C₀ to C₇ may be generated. But we know that extra Carry may also be generated, which we term as C₈.
- If C₈ is generated, i.e., Carry is generated, reflecting our Carry flag, which results in the C flag or C status bit being set to 1.

Status Bit condition



Let's move further to the last two carries, C₇ and C₈. If the XOR of these two carries comes out to be 1, then we can say that the Overflow condition has happened and the V flag is set to 1; otherwise, it's set to 0. This is the most occurring case when we have negative numbers represented in 2's complement form.

Example

- Let's add **+70** and **+80** using 8-bit 2's complement.

Step 1: Represent Numbers in Binary

Decimal	8-bit Binary
+70	01000110
+80	01010000

Step 2: Add the Numbers

```
01000110 (70)
+01010000 (80)
-----
10010110 (150)
(Result = -106 in 2's complement!)
```

Contd...

- Step 3: Identify C7 and C8

Let's break down carry generation:

Bits 6 to 7 (second highest to highest bit):

Bit 6: $1 + 0 = 1 \rightarrow$ No carry into bit 7

Bit 7: $0 + 0 + 0$ (previous carry) $= 0 \rightarrow$ No carry out

But in reality (in full adder), final carry out = 1 (due to full bitwise addition)

So: C7 (carry into bit 7) = 1

C8 (carry out from bit 7) = 0

- $\Rightarrow C7 \oplus C8 = 1 \rightarrow$ Overflow happened
- V flag = 1
- Wrong sign in result: two positives yielded a negative $\rightarrow$ Overflow detected

Example 2: Lets add +20 and -10

- Step 1: Binary Representation

Decimal to 8-bit Binary Conversion:

+20 $\rightarrow$ 00010100

-10 $\rightarrow$ 11110110

Step 2: Perform Binary Addition

00010100 (+20)

+11110110 (-10)

00001010 (Result = +10)

Step 3: Detect Overflow using C7 and C8

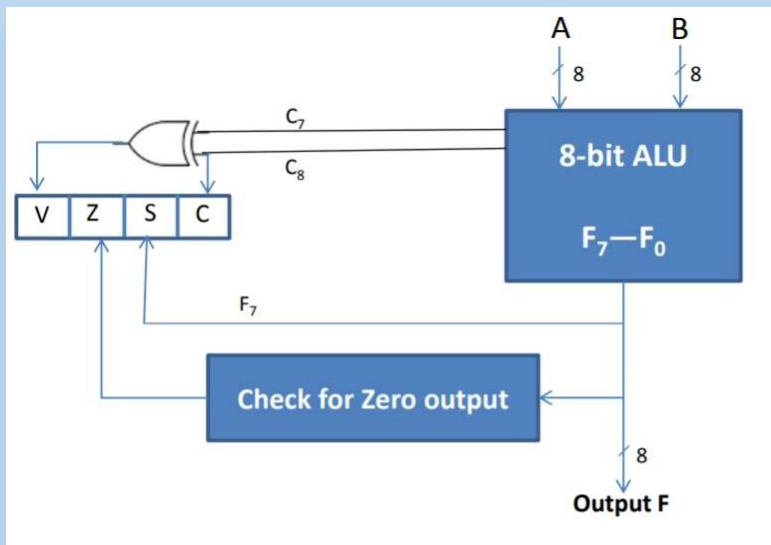
Carry into MSB (C7) = 1

Carry out of MSB (C8) = 1

$C7 \oplus C8 = 0 \rightarrow \checkmark$ No Overflow

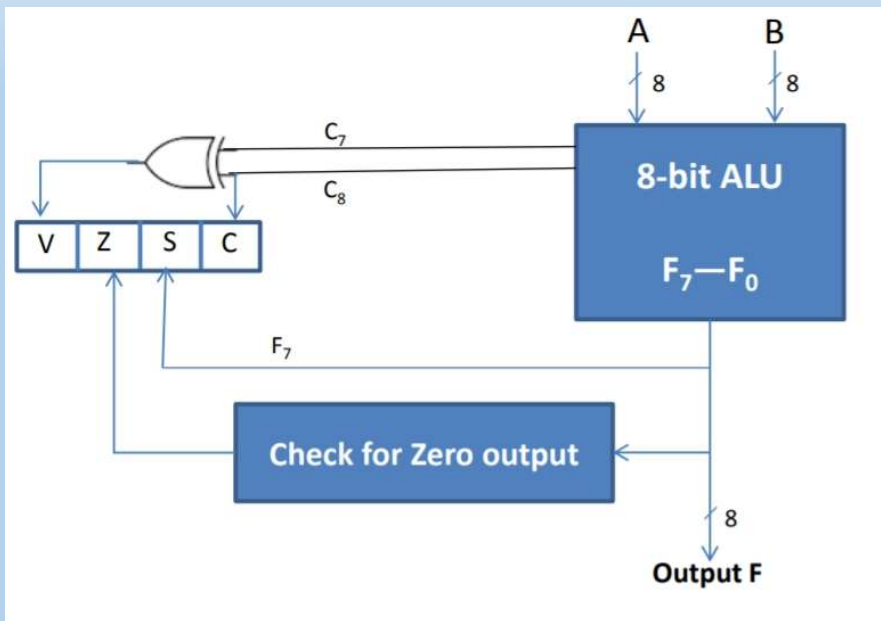
V flag = 0

Status Bit condition



Now moving on to the next flag which is the sign flag(S), the sign flag is set to 1 or 0 based on the output of the 8-bit ALU. As we know if the number is positive, then the most significant bit of a number is represented to be 0, which means if F7 is 0 then we can say that the number is positive and if F7 is 1 then we say that the number is negative.

Status Bit condition



Lastly, we have a Zero flag, Zero status bit Z, "Z" is said to be 1 if the output of all the bits from F₀ to F₇ is 0, then we can say that the zero flag is SET.

Now all these four bits that are "V", "Z", "S" and "C" are reflected based on the arithmetic or logic operation carried out on the 8-bit ALU.

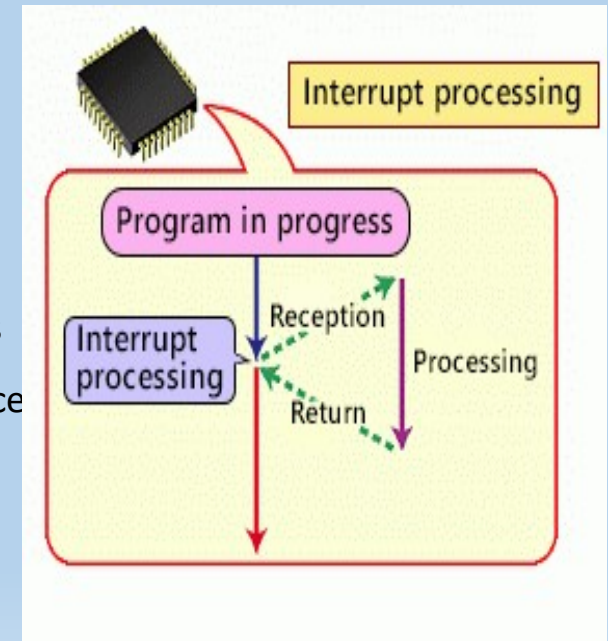
Interrupt

- The interrupt is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process. In I/O devices one of the bus control lines is dedicated for this purpose and is called the *Interrupt Service Routine (ISR)*.

Interrupt processing

Whenever an interrupt occurs, it causes the CPU to stop executing the current program. Then, comes the control to interrupt handler or interrupt service routine.

- These are the steps in which ISR handles interrupts. These are as follows
- **Step 1** When an interrupt occurs let assume processor is executing i^{th} instruction and program counter will point to the next instruction $(i+1)^{\text{th}}$.
- **Step 2** When an interrupt occurs the program value is stored on the process stack and the program counter is loaded with the address of interrupt service routine.
- **Step 3** Once the interrupt service routine is completed the address on the process stack is popped and placed back in the program counter.
- **Step 4** Now it executes the resume for $(i+1)^{\text{th}}$ line.



Types of Interrupt

- There are two types of interrupts which are as follows:
 - Hardware Interrupt
 - Software Interrupt

Hardware interrupts

The interrupt signal generated from external devices and i/o devices are made interrupt to CPU when the instructions are ready.

For example In a keyboard if we press a key to do some action this pressing of the keyboard generates a signal that is given to the processor to do action, such interrupts are called hardware interrupts.

Hardware interrupts are classified into two types which are as follows :

- **Maskable Interrupt** :The hardware interrupts that can be delayed when a highest priority interrupt has occurred to the processor.
- **Non Maskable Interrupt**: The hardware that cannot be delayed and immediately be serviced by the processor.

Software interrupts

- The interrupt signal generated from internal devices and software programs need to access any system call then software interrupts are present.
- Software interrupt is divided into two types. They are as follows:

Normal Interrupts : The interrupts that are caused by the software instructions are called normal software instructions.

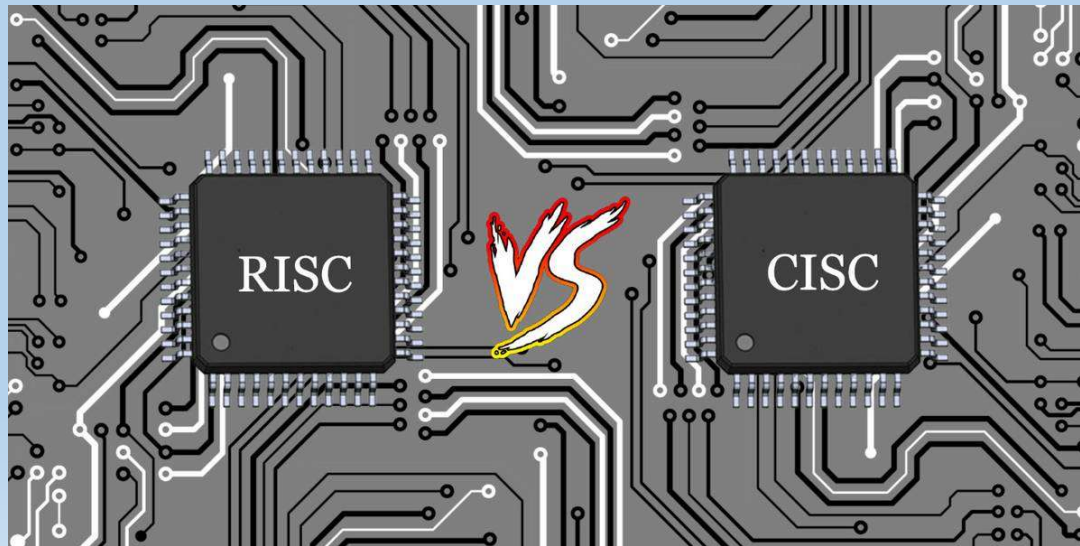
Exception : Exception is nothing but an unplanned interruption while executing a program. For example :while executing a program if we got a value that is divided by zero is called an exception

Instruction Set Architecture

- An Instruction Set Architecture (ISA) is part of the abstract model of a computer that defines how the CPU is controlled by the software. The ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done.
- The ISA provides the only way through which a user is able to interact with the hardware. It can be viewed as a programmer's manual because it's the portion of the machine that's visible to the assembly language programmer, the compiler writer, and the application programmer.

Contd...

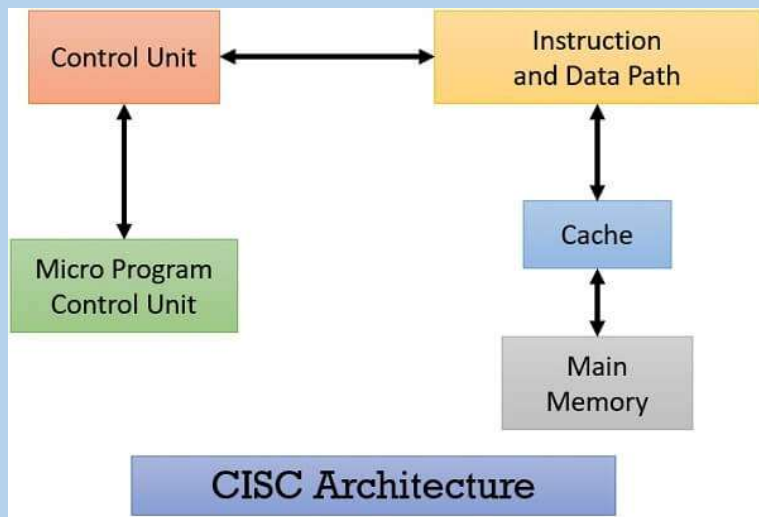
- The two major instruction sets architectures
 - 1) CISC => Complex Instruction Set Computing
 - 2) RISC => Reduced Instruction Set Computing



CISC

- CISC is an acronym for Complex Instruction Set Computer and are chips that are easy to program and which make efficient use of memory.
- Since the earliest machines were programmed in assembly language and memory was slow and expensive, the CISC philosophy made sense, and was commonly implemented in such large computers as the PDP-11 and the DECsystem 10 and 20 machines.
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.

CISC Architecture



- CISC architectures directly use the memory, instead of a register file. The above alongside shows the architecture of CISC with micro programmed control and cache memory.
- This architecture uses cache memory for holding both data and instructions.
- Thus, they share the same path for both instructions and data. CISC has instructions with variable length format.
- Thus, the number of clock cycles required to execute the instructions may be varied.
- Instructions in CISC are executed by micro program which has sequence of microinstructions.

Contd...

- CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor.
- For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this.
- But recent changes in software and hardware technology have forced a re-examination of CISC and many modern CISC processors are hybrids, implementing many RISC principles.

CISC

- **The major characteristics of CISC architecture**
- A large number of instructions— typically from 100 to 250 instructions
- Some instructions that perform specialized tasks and are used infrequently
- A large variety of addressing modes—typically from 5 to 20 different modes
- Variable-length instruction formats
- Instructions that manipulate operands in memory

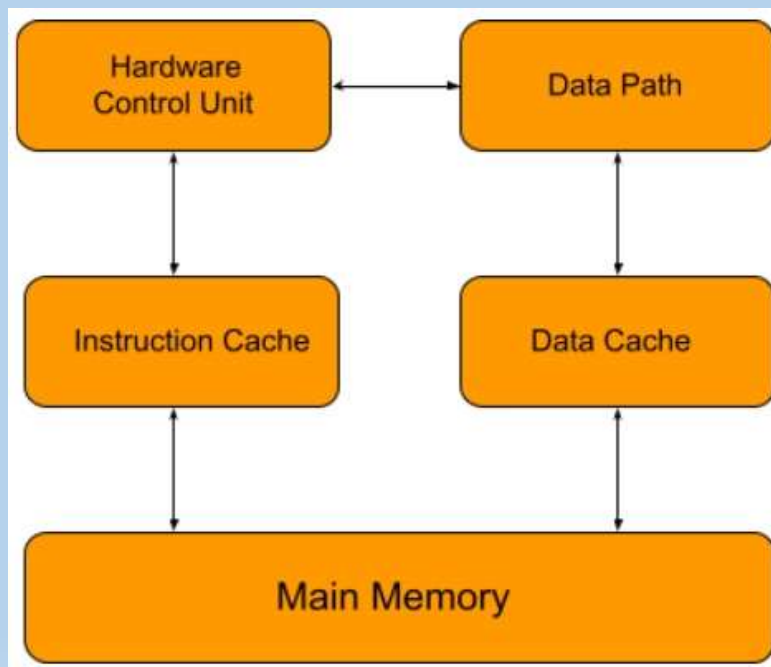
RISC

- RISC (reduced instruction set computer) is a microprocessor that is designed to perform a smaller number of types of computer instructions so that it can operate at a higher speed (perform more millions of instructions per second, MIPS)
- Although CISC reduces usage of memory and compiler, it requires more complex hardware to implement the complex instructions.
- The speed of the execution is increased by using smaller number of instructions .This uses pipeline technique for execution of any instruction.

Contd...

- The main idea behind using RISC is to simplify hardware by using an instruction set composed of a few basic steps for loading, evaluating, and storing operations just like a load command will load data, a store command will store the data.
- Reduced Instruction Set Computer (RISC), is a type of computer architecture that operates on a small, highly optimized set of instructions, instead of a more specialized set of instructions, which can be found in other types of architectures. This architecture means that the computer microprocessor will have fewer cycles per instruction.

Architecture



The figure is the architecture of RISC processor, which uses separate instruction and data caches and their access paths also different. There is one instruction per machine cycle in RISC processor.

Example

- Let us see an example : Adding of two numbers can be as shown below.
LOAD A, 2:3
LOAD B, 5:2
SUM A, B
STORE 2:3, A
- From the above it can be seen that Addition requires four steps in RISC processor
- Though many instructions were required in RISC, time taken by these instructions to complete execution is same as time required to execute “ADD” in CISC.
- The design of the control unit is also simple due to the limited number of instructions.

Contd...

- RISC processors/architectures are used across a wide range of platforms nowadays, ranging from tablet computers to smartphones, as well as supercomputers (i.e. Summit top500 list in 2018).
- This architecture include alpha, AVR, ARM, PIC, PA-RISC, and power architecture.

RISC

- **The major characteristics of a RISC architecture**
 - Relatively few instructions
 - Relatively few addressing modes
 - Memory access limited to load and store instructions
 - All operations done within the registers of the CPU
 - Fixed-length, easily decoded instruction format
 - Single-cycle instruction execution
 - Hardwired rather than micro-programmed control
 - Large number of register set

CISC and RISC Convergence

- State of the art processor technology has changed significantly since RISC chips were first introduced in the early '80s. Because a number of advancements are used by both RISC and CISC processors, the lines between the two architectures have begun to blur. In fact, the two architectures almost seem to have adopted the strategies of the other.
- Because processor speeds have increased, CISC chips are now able to execute more than one instruction within a single clock. This also allows CISC chips to make use of pipelining. With other technological improvements, it is now possible to fit many more transistors on a single chip.

- This gives RISC processors enough space to incorporate more complicated, CISC-like commands. RISC chips also make use of more complicated hardware, making use of extra function units for superscalar execution.
- All of these factors have led some groups to argue that we are now in a "post-RISC" era, in which the two styles have become so similar that distinguishing between them is no longer relevant.
- However, it should be noted that RISC chips still retain some important traits. RISC chips strictly utilize uniform, single-cycle instructions. They also retain the register-to-register, load/store architecture. And despite their extended instruction sets, RISC chips still have a large number of general purpose registers.

BERKELEY RISC I

- The RISC 1 computer implemented in the late 1970s used a very large register set of 138×32 bit registers.
- These were arranged in eight overlapping windows of 24 registers each. Each window was split so that six registers could be used for parameter passing during subroutine calls. A pointer was simply changed to select the group of six registers.
- To perform a basic call or return simply needed a change of pointer. The large number of registers is needed to minimize the number of fetches to the outside world.
- With this simple window technique, procedure calls can be performed extremely quickly. This can be very beneficial for real-time applications where fast responses are necessary.

Overlapped Register Windows

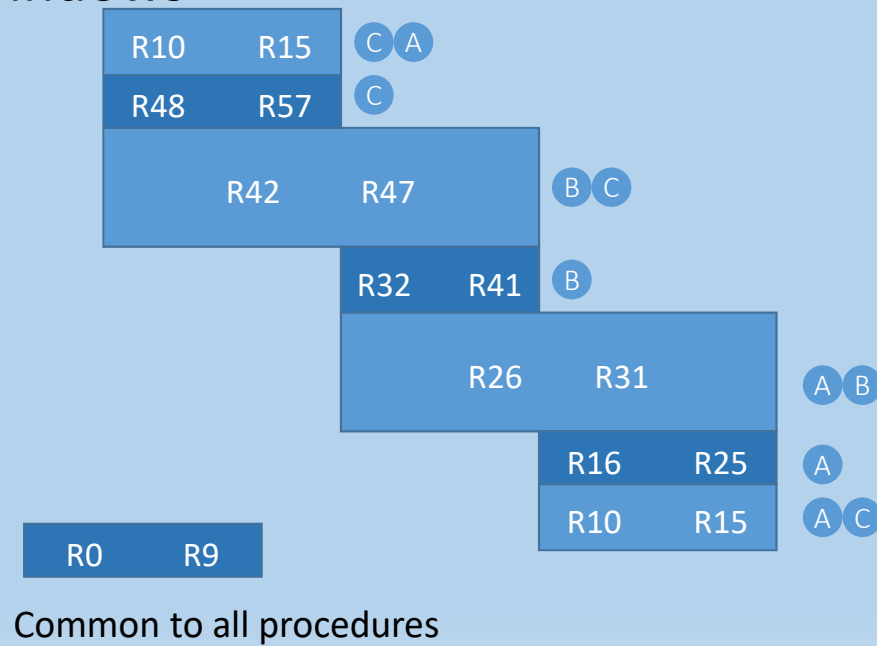
- In procedure call and return process
 - Saving and restoring registers along with passing of parameters and results involve time consuming operations
 - Some system use multiple register banks with each procedure allocated its own bank of registers
 - Other systems use memory stack to store the context that are needed by the procedure which involves many memory access

Overlapped Register Windows

- It provides passing of parameters and avoid the need for saving and restoring register values
- Each new procedure allocated with new window consisting of a set of registers
- Windows of adjacent procedures have overlapping registers; to share the parameters and results

Example

- Overlapped Register Windows



Assignment

- Explain component of CPU.
- Explain register organization in CPU with its control word format.
- Explain design methods for ALU.
- Explain Addressing modes with example.
- Evaluate following with 3,2,1,0- address instructions.
 - $Y = (A - B) / (C / D + E / F - G / H)$
 - $Y = (A + B)^2 - (C - D)^3$
 - $Y = ((A + (B / C) - (D * E / F)) / (G - H / I)) - (J + K - (L / M))$
- Explain different types of instruction with examples.
- Differentiate between RISC and CISC.
- Explain Overlapped register windows.
- Explain in detail about 64-bit processor with alu, register organization, instructions and addressing.

End Of Class